NAME: S.DEVI SHREE

REG.NO: 192511149

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization

(1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

1.For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

FDs:

- SID → SName
- CID → CName, Instructor
- SID, CID → Grade

Candidate key: (SID, CID) — a student's grade for a course is uniquely identified only by the (student, course) pair; neither SID nor CID alone determines Grade.

2.Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

UNF → 1NF: Step-by-Step Demonstration

Unnormalized relation (UNF) — contains a repeating/multivalued group:

SID	SName	Courses
S1	Alice	{C101, C102}
S2	Bob	{C103}

This violates 1NF because the Courses cell is **non-atomic** (a set of values in one cell).

Step-by-step conversion:

1. **Identify the repeating group** — Courses holds multiple values per student.
2. **Flatten it** — repeat the key (SID, SName) for each value, giving one course per row.
3. **Verify atomicity** — every cell now holds exactly one indivisible value.

Result (1NF):

SID	SName	Course
S1	Alice	C101
S1	Alice	C102
S2	Bob	C103

Justification: Each attribute now holds a single atomic value, and the relation has a well-defined key (SID, Course) — satisfying 1NF.

3.Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.

R(A,B,C,D,E), FDs: A→BC, BC→D, D→E → 2NF

Step 1 — Find the candidate key (compute closure):
A+ = {A} → apply A→BC → {A,B,C} → apply BC→D → {A,B,C,D} → apply D→E → {A,B,C,D,E} = all attributes.

So A+ = all attributes → **A alone is the candidate key**.

Step 2 — Check for partial dependency: Partial dependency can only exist when the key is **composite** and a non-key attribute depends on part of it. Since the key here is the single attribute A, **no partial dependency is possible** — the relation is automatically in **2NF**.

Step 3 — Note for full normalization: Although 2NF is already satisfied, the chain A → BC → D → E contains **transitive dependencies** (B,C → D and D → E don't involve A directly), which would need removal for 3NF. Applying the standard FD-synthesis decomposition:

Table	Attributes	Key
R1	A, B, C	A
R2	B, C, D	(B,C)
R3	D, E	D

This is dependency-preserving and lossless (each table's key is the LHS of the FD that produced it), and goes beyond 2NF into 3NF.

4.Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) → 3NF

Candidate key: EmpID

FDs:

- EmpID → EmpName, DeptID
- DeptID → DeptName, ManagerID (transitive — each department has one manager)

Transitive dependency: EmpID → DeptID → DeptName, ManagerID. DeptName and ManagerID don't depend directly on EmpID; they depend on DeptID, a non-key attribute.

Decomposition to 3NF:

Table	Attributes	Key
Employee	EmpID, EmpName, DeptID	EmpID
Department	DeptID, DeptName, ManagerID	DeptID

No transitive dependency remains → **3NF achieved**.

5.Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

R(A,B,C,D), FDs: AB→C, C→D, D→A → Verify/Decompose to BCNF

Step 1 — Find candidate key(s):
AB+ = {A,B} → AB→C → {A,B,C} → C→D → {A,B,C,D} = all. So AB+ = all attributes. Check subsets: A+ = {A}, B+ = {B} — neither alone works. So **AB is the (only) candidate key**.

Step 2 — Check BCNF: every FD's LHS must be a superkey.

- AB→C: AB is the key → OK.
- C→D: is C a superkey? C+ = {C,D,A} (via C→D→A) — missing B → **not a superkey → violates BCNF**.
- D→A: D+ = {D,A} — missing B,C → **not a superkey → violates BCNF**.

Step 3 — Decompose using the violating FD C→D:
R1 = (C,D), R2 = R − D = (A,B,C)

Step 4 — Re-check R2(A,B,C): From C→D→A (transitivity), C→A holds and projects onto R2. Is C a superkey of R2? C+ within R2 = {C,A} — missing B → **still violates BCNF**.

Decompose R2 using C→A:
R3 = (A,C), R4 = R2 − A = (B,C)

Final BCNF decomposition:

Table	Attributes	Key	Check
R1	C, D	C	C→D, C is key → BCNF ✓

Table	Attributes	Key	Check
R3	A, C	C	C→A, C is key → BCNF ✓
R4	B, C	(B,C)	no nontrivial FD → BCNF ✓

Lossless check: R3 ⋈ R1 (common attribute C) → (A,C,D); then ⋈ R4 (common attribute C) → (A,B,C,D) — reconstructs the original relation exactly.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Experimental Verification of Lossless-Join Decomposition (Chase/Tabular Algorithm)

Setup: R(A,B,C), FD: B → C, decomposed into R1(A,B) and R2(B,C).

Step 1 — Build the tableau (one row per decomposed relation, one column per attribute of R). Use the **unsubscripted symbol a_j** if the attribute belongs to that relation, else a **unique subscripted symbol b_{ij}**:

	A	**B**	**C**
Row1 (R1)	a_1	a_2	b_{13}
Row2 (R2)	b_{21}	a_2	a_3

Step 2 — Apply each FD. For B→C: both rows agree on column B ($a_2 = a_2$), so their C-values must be equalized. Row2's C is the unsubscripted a_3, so set Row1's C to a_3 as well:

	A	**B**	**C**
Row1	a_1	a_2	**a_3**
Row2	b_{21}	a_2	a_3

Step 3 — Check for a fully unsubscripted row. Row1 is now (a_1, a_2, a_3) — entirely unsubscripted, matching the original relation's row exactly.

Conclusion: Since a completely "all-a" row was produced, the decomposition {R1(A,B), R2(B,C)} is **lossless-join** with respect to B→C. (If no row had become fully unsubscripted after applying all FDs to closure, the decomposition would be **lossy**.)

7. Identify the multi-valued dependencies in TEACHER(TID, Subject,

Hobby) and decompose it into 4NF.

TEACHER(TID, Subject, Hobby) → Identify MVDs and Decompose to 4NF

Analysis: A teacher can teach **multiple subjects** and have **multiple hobbies**, and these two facts are **independent** of one another (a teacher's hobbies have nothing to do with which subjects they teach).

MVDs:

- TID →→ Subject
- TID →→ Hobby

Why this causes redundancy: If teacher T101 teaches 2 subjects and has 3 hobbies, storing them in one relation forces **2 × 3 = 6 rows** to represent facts that could be captured in 2 + 3 = 5 rows across two tables — pure redundancy with no FD justifying it.

Example:

TID	**Subject**	**Hobby**
T101	Math	Chess
T101	Math	Painting
T101	Physics	Chess
T101	Physics	Painting

Decomposition to 4NF:

Table	**Attributes**	**Key**
TeacherSubject	TID, Subject	(TID, Subject)
TeacherHobby	TID, Hobby	(TID, Hobby)

Each independent multivalued fact is now isolated → **4NF achieved**.

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Join Dependencies → Redundancy → Decompose to 5NF (PJNF)

Classic example — Supplier/Part/Project: SPJ(Supplier, Part, Project)

Business rule (the join dependency): If supplier S supplies part P, **and** part P is used in project J, **and** supplier S supplies to project J, **then** S actually supplies P specifically for J. Formally: JD(SP, PJ, SJ).

Redundancy arises: A single ternary table forces you to store one row per valid (S,P,J) combination even when the underlying facts are really three separate pairwise relationships. Any inconsistency between the three pairwise facts (e.g., adding "supplier supplies part" without a matching "part used in project + supplier supplies project") creates anomalies that no FD or MVD alone captures — only a **3-way JD** describes the constraint.

Decomposition to 5NF:

Table	Attributes
SP	Supplier, Part
PJ	Part, Project
SJ	Supplier, Project

Verification: SP ⋈ PJ ⋈ SJ must reproduce exactly the original valid (S,P,J) triples, with **no spurious combinations** — this is checked by confirming the JD is satisfied by the instance (using the Chase test). Since no two of the three projections alone can losslessly reconstruct R, this is a genuine 5NF (PJNF) decomposition — the relation cannot be decomposed further without loss.

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Poorly Designed College Schema → Identify Anomalies → 3NF

Assumed schema: CollegeRecord(StudentID, StudentName, DeptID, DeptName, HOD, CourseID, CourseName, Marks)

FDs:

- StudentID → StudentName, DeptID
- DeptID → DeptName, HOD (transitive)
- CourseID → CourseName (transitive)
- StudentID, CourseID → Marks

Anomalies:

- **Insertion:** A new department (with its HOD) cannot be recorded until at least one student is enrolled in it.
- **Update:** If the HOD changes, every row of every student in that department must be updated — high risk of inconsistent data if even one row is missed.
- **Deletion:** If the last student in a department is removed from the table, all record of that department and its HOD disappears.

Decomposition to 3NF:

Table	Attributes	Key
Student	StudentID, StudentName, DeptID	StudentID
Department	DeptID, DeptName, HOD	DeptID
Course	CourseID, CourseName	CourseID
Marks	StudentID, CourseID, Marks	(StudentID, CourseID)

All transitive dependencies removed → **3NF achieved**.

10.Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Compute Attribute Closures and Determine All Candidate Keys

Example relation: R(A,B,C,D,E) with FDs: A→B, BC→D, D→E, E→C

Step 1 — Try single attributes:
A+: start {A} → apply A→B → {A,B}. No other FD's LHS (BC, D, E) is fully contained in {A,B} → **A+ = {A,B}** — incomplete, not a key.

Step 2 — Try E:
E+: start {E} → apply E→C → {E,C}. BC needs B, not present → stuck. **E+ = {E,C}** — incomplete.

Step 3 — Try pair AC:
AC+: start {A,C} → A→B gives {A,B,C} → BC→D (B,C present) gives {A,B,C,D} → D→E gives {A,B,C,D,E} = **all attributes**.
Check minimality: A+ alone = {A,B} (incomplete), C+ alone = {C} (no FD starts with just C) — incomplete. So **AC is a minimal candidate key**.

Step 4 — Try pair AE:
AE+: start {A,E} → A→B gives {A,B,E} → E→C gives {A,B,C,E} → BC→D (B,C present) gives {A,B,C,D,E} = **all attributes**.
Check minimality: A+ = {A,B} (incomplete), E+ = {E,C} (incomplete). So **AE is also a minimal candidate key**.

Conclusion: This relation has **two candidate keys**: AC and AE (this happens because E→C, so E can "stand in" for C in generating the same closure). Neither is redundant since one cannot derive E from C (no C→E given), so both remain distinct minimal keys.

General method used (applies to any relation): For each candidate subset X of attributes, compute X+ by repeatedly adding the RHS of any FD whose LHS ⊆ current closure, until no more attributes can be added. If X+ = all attributes of R **and no proper subset of X has this property**, X is a candidate key.